LAPORAN PRAKTIKUM STRUKTUR DATA

Stack

Oleh:

Naufal Arkan Octyandi

2511533005

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU: DR. WAHYUDI, S.T, M.T

ASISTEN LABOR: SHERLY SUKMADIRA

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 17 APRIL 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data yang berjudul “Stack” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum serta untuk menambah wawasan dan pemahaman mengenai konsep struktur data, khususnya penggunaan Stack. Dalam praktikum ini, penulis mempelajari bagaimana cara menyimpan data dengan prinsip LIFO (*Last In First Out*), serta melakukan operasi dasar seperti *push*, *pop*, dan *peek* menggunakan Stack dalam bahasa pemrograman Java.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca dan dapat menjadi referensi tambahan dalam mempelajari struktur data, khususnya konsep Stack.

Padang, 2026

Tim Penyusun

Naufal Arkan Octyandi

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 LATAR BELAKANG 1
1.2 TUJUAN PRAKTIKUM 1
1.3 MANFAAT PRAKTIKUM 1
BAB II PEMBAHASAN PRAKTIKUM 2
2.1 STACK 2
BAB III KESIMPULAN 6
3.1 KESIMPULAN 6
DAFTAR PUSTAKA 7

BAB I
PENDAHULUAN

1.1 LATAR BELAKANG

Dalam perkembangan teknologi informasi, pengolahan data yang efisien menjadi hal yang sangat penting dalam dunia pemrograman. Salah satu cara untuk mengelola data dengan baik adalah dengan menggunakan struktur data yang sesuai. Struktur data berperan dalam mengatur bagaimana data disimpan, diakses, dan dimanipulasi agar program dapat berjalan lebih optimal.

Salah satu struktur data yang sering digunakan adalah Stack, yaitu struktur data yang bekerja dengan prinsip LIFO (*Last In First Out*), di mana data yang terakhir masuk akan menjadi data pertama yang keluar. Konsep ini banyak digunakan dalam berbagai aplikasi, seperti pengelolaan memori, undo-redo pada aplikasi, serta evaluasi ekspresi dalam pemrograman.

Oleh karena itu, melalui praktikum ini, penulis mempelajari konsep dasar Stack serta implementasinya menggunakan bahasa pemrograman **Java**, sehingga diharapkan dapat memahami cara kerja dan penerapannya dalam pengolahan data.

1.2 TUJUAN PRAKTIKUM

1. Memahami konsep dasar struktur data Stack.
2. Mengetahui prinsip kerja LIFO (*Last In First Out*).
3. Mempelajari operasi dasar Stack seperti *push*, *pop*, dan *peek*.

1.3 MANFAAT PRAKTIKUM

1. Menambah pemahaman tentang struktur data Stack.
2. Melatih kemampuan dalam pemrograman menggunakan Java.
3. Membantu memahami cara pengelolaan data secara efisien.

BAB II
PEMBAHASAN PRAKTIKUM

2.1 STACK

Kita akan membuat kelas NilaiMaksimum dengan menggunakan struktur data Stack untuk mencari nilai maksimum dari elemen yang ada di dalam stack. Method max() bekerja dengan cara mengambil semua elemen dari stack satu per satu menggunakan pop(), lalu membandingkannya untuk mendapatkan nilai terbesar. Selama proses ini, elemen disimpan sementara ke stack cadangan agar data tidak hilang. Setelah selesai, semua elemen dikembalikan ke stack awal seperti semula. Di main program, kita menambahkan beberapa angka ke dalam stack, menampilkan isi stack, menampilkan elemen teratas (peek), dan menampilkan nilai maksimum.

```java
package pekan3_2511533005;
import java.util.Stack;
public class NilaiMaksimum_2511533005 {
    public static int max(Stack<Integer> s) {
        Stack<Integer> backup_3005 = new Stack<Integer> ();
        int maxValue = s.pop();
        backup_3005.push (maxValue);
        while (!s.isEmpty()) {
            int next= s.pop();
            backup_3005.push(next);
            maxValue = Math.max (maxValue, next);
        }
        while (!backup_3005.isEmpty()) {
            s.push (backup_3005.pop());
        }
        return maxValue;
    }

    public static void main(String[] args) {
        Stack<Integer> s = new Stack <Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack " +s);
        System.out.println("Stack Teratas "+ s.peek());
        System.out.println("nilai maksimum "+ max(s));

    }

}
```

Selanjutnya kita akan membuat kelas baru bernama Siswa. Program ini membuat implementasi Stack secara manual menggunakan ArrayList untuk menyimpan objek Siswa. Class Siswa berfungsi sebagai model data (nama dan NIM), sedangkan class SiswaStack berfungsi sebagai pengelola stack dengan operasi seperti push (tambah data), pop (hapus data teratas), peek (lihat data teratas), dan isEmpty.

Pada main, program langsung digunakan sebagai driver (tidak ada class terpisah), di mana beberapa data siswa dimasukkan ke stack, kemudian ditampilkan, diambil elemen teratasnya, dan dihapus satu per satu sesuai konsep LIFO.

```java
package pekan3_2511533005;
import java.util.ArrayList;
class Siswa_2511533005 {
    String nama_3005;
    int nim_3005;

    public Siswa_2511533005(String nama_3005, int nim_3005) {
        this.nama_3005= nama_3005;
        this.nim_3005= nim_3005;
    }

    @Override
    public String toString() {
        return "Nim:" + nim_3005 + ", Nama: "+ nama_3005;
    }

}
public class SiswaStack_2511533005 {
    private ArrayList<Siswa_2511533005> stack;

    public SiswaStack_2511533005() {
        stack= new ArrayList<> ();
    }

    public void push(Siswa_2511533005 mhs) {
        stack.add(mhs);
    }

public Siswa_2511533005 pop() {
    if (!isEmpty()) {
        return stack.remove (stack.size() -1);
    }
    return null;
}

```

```java
public Siswa_2511533005 peek() {
    if (!isEmpty()) {
        return stack.get(stack.size() -1);
    }
    return null;
}

public boolean isEmpty() {
    return stack.isEmpty();
}
public void tampilkanSiswa() {
    for (int i = stack.size() - 1; i >= 0; i--) {
        System.out.println(stack.get(i));
    }
}
public static void main(String[] args) {
    SiswaStack_2511533005 studentStack = new SiswaStack_2511533005();

    Siswa_2511533005 mhs1 = new Siswa_2511533005 ("Ali", 1);
    Siswa_2511533005 mhs2 = new Siswa_2511533005 ("Bobby", 2);
    Siswa_2511533005 mhs3 = new Siswa_2511533005 ("Charles", 3);

    studentStack.push(mhs1);
    studentStack.push(mhs2);
    studentStack.push(mhs3);

    System.out.println("Siswa didalam Stack: ");
    studentStack.tampilkanSiswa();

    System.out.println("siswa teratas " + studentStack.peek());
    System.out.println("mengeluiarkan siswa teratas dari stack: " + studentStack.pop());
    System.out.println("daftar siswa setelah di pop: ");
    studentStack.tampilkanSiswa();
}
}
```

Class stackArray berfungsi sebagai pengelola stack yang berisi atribut array sebagai tempat penyimpanan data dan variabel top sebagai penunjuk posisi elemen teratas. Class ini menyediakan operasi dasar stack seperti push untuk menambah elemen, pop untuk menghapus elemen teratas, peek untuk melihat elemen teratas, isEmpty untuk mengecek apakah stack kosong, serta print untuk menampilkan isi stack.

Selain itu, terdapat penanganan kondisi overflow (jika stack penuh) dan underflow (jika stack kosong).

```java
package pekan3_2511533005;

public class stackArray_2511533005 {
    static final int MAX_3005=1000;
    int top;
    int a[] = new int[MAX_3005];
    boolean isEmpty_3005()
    {
        return (top <0);
    }
    stackArray_2511533005()
    {
        top=-1;
    }
    boolean push_3005(int x)
    {
        if (top >= (MAX_3005 -1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a[++top] = x;
            System.out.println(x + " Dimasukkan dalam stack");
            return true;
        }
    }
    int pop_3005()
    {
        if (top<0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
```

```java
            return 0;
        }
        else {
            int x = a[top--];
            return x;
        }
    }
    int peek_3005()
    {
        if (top <0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;
        }
    }
    void print_3005 () {
        for(int i = top; i >-1; i--) {
            System.out.print(" " + a[i]);
        }
    }
}
```

Lalu selanjutnya kita bikin class drivernya. class stackArrayDriver berperan sebagai driver atau program utama yang digunakan untuk menjalankan dan menguji fungsi-fungsi pada class stackArray. Pada bagian ini dilakukan beberapa operasi seperti menambahkan elemen ke stack, menghapus elemen, melihat elemen teratas, dan menampilkan seluruh isi stack.

```java
package pekan3_2511533005;

public class stackArrayDriver_2511533005 {

    public static void main(String[] args) {
        stackArray_2511533005 s = new stackArray_2511533005();
        s.push_3005(10);
        s.push_3005(20);
        s.push_3005(30);
        System.out.println(s.pop_3005() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah:" + s.peek_3005());
        System.out.println("Elemen pada stack:");
        s.print_3005();
    }
}
```

Kita membuat class dengan struktur data Stack untuk menghitung hasil dari ekspresi postfix (Reverse Polish Notation). Method postfixEvaluate() membaca ekspresi dalam bentuk string, lalu memprosesnya menggunakan Scanner.

Jika yang dibaca adalah angka (operand), maka akan dimasukkan ke dalam stack (push). Jika yang dibaca adalah operator, maka program akan mengambil dua nilai teratas dari stack (pop), melakukan operasi sesuai operator (penjumlahan, pengurangan, perkalian, atau pembagian), lalu hasilnya dimasukkan kembali ke stack. Proses ini dilakukan sampai semua ekspresi selesai, dan hasil akhir diambil dari stack. Pada bagian main, program langsung menjalankan fungsi tersebut dengan contoh ekspresi postfix dan menampilkan hasilnya.

```java
import java.util.Stack;
import java.util.Scanner;
public class StackPostfix_2511533005 {
    public static int postfixEvaluate (String expression) {
        Stack<Integer> s= new Stack <Integer> ();
        Scanner input_3005= new Scanner (expression);
        while (input_3005.hasNext()) {
            if(input_3005.hasNextInt()) {   // an operand (integer)
                s.push(input_3005.nextInt());
            } else {                        // an operator
                String operator = input_3005.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator.equals(" + ")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);
            }
        }
        }
        input_3005.close();
        return s.pop();
    }

    public static void main(String[] args) {
        System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7 -"));

    }

}
```

BAB III
KESIMPULAN

3.1 KESIMPULAN

Berdasarkan praktikum yang telah dilakukan menggunakan Java, dapat disimpulkan bahwa struktur data Stack merupakan salah satu metode penyimpanan data yang bekerja dengan prinsip LIFO (*Last In First Out*), di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

Dalam praktikum ini, Stack diimplementasikan dengan berbagai cara, yaitu menggunakan class bawaan Java (Stack), menggunakan ArrayList, serta menggunakan array secara manual. Setiap metode memiliki cara kerja yang sama, namun berbeda dalam pengelolaan dan tingkat kompleksitas implementasinya.

Selain itu, praktikum juga menunjukkan penerapan Stack dalam berbagai kasus, seperti pengelolaan data sederhana (data siswa), pencarian nilai maksimum tanpa menghilangkan isi stack, serta evaluasi ekspresi postfix. Hal ini membuktikan bahwa Stack memiliki peran penting dalam pemrograman, khususnya dalam pengolahan data dan penyelesaian masalah yang membutuhkan urutan proses tertentu.

Dengan demikian, pemahaman tentang Stack sangat penting sebagai dasar dalam mempelajari struktur data yang lebih kompleks serta dalam pengembangan program yang efisien dan terstruktur.

DAFTAR PUSTAKA

Programiz. *Java Stack*.
Tersedia pada: https://www.programiz.com/java-programming/stack
GeeksforGeeks. *Stack in Java*.
Tersedia pada: https://www.geeksforgeeks.org/stack-class-in-java/